

1.LangGraph를 3일차에서 보는 이유

LangGraph를 3일차에서 보는 이유

Tool이란 무엇인가?

Tool은 개발자가 사용하는 일반적인 도구가 아니라, Agent가 업무를 수행하기 위해 호출할 수 있는 기능 단위입니다.

- 설비 상태 조회
- 알람 이력 확인
- 공정 상태 확인
- 품질 지표 조회
- 정비 이력 확인
- 기술 문서 검색

LangGraph의 역할

MCP가 Tool의 규격과 연결 방식을 정리한다면, LangGraph는 그 Tool들을 어떤 조건과 순서로 호출할지 설계합니다.

- ① 설비 반복 알람 분석 사례에서는 설비 ID와 알람 코드가 있으면 DB/Log Tool을 호출하고, 조치 절차나 장애 대응 가이드가 필요하면 RAG Tool을 호출하는 Routing 기준이 필요합니다.

3일차에서는 이 흐름을 통해 Tool 선택, State 전달, 실행 기록을 어떻게 설계할지 확인합니다.

multi_agent_roles.py를 LangGraph 관점으로 해석하기

multi_agent_roles.py는 실제 LangGraph 구현 파일이라기보다, LangGraph 관점으로 해석할 수 있는 Multi-Agent 흐름 예제입니다. 중요한 것은 코드 문법이 아니라 아키텍처 해석 관점입니다.

1

CoordinatorAgent

설비 ID, 알람 코드, 증상 같은 핵심 정보를 추출하는 요청 분석 Node

2

DBInvestigationAgent

설비 상태, 알람 이력, 공정 상태, 품질 지표, 정비 이력 등 DB/Log Tool 호출 Node

3

ManualRAGAgent

search_manual을 호출해 기술 문서와 장애 대응 가이드를 찾는 RAG Tool Node

4

IncidentSummaryAgent

DB 근거와 RAG 근거를 종합해 final_summary를 만드는 Summary Node

multi_agent_roles.py — 아키텍처 해석 핵심

State와 agent_steps

State는 Node 간에 공유되는 작업 정보이고, agent_steps는 어떤 단계가 어떤 순서로 실행되었는지 보여주는 실행 흐름 기록입니다.

- ❏ 각 Agent를 독립된 이름으로 보는 것이 아니라, 어떤 Node가 어떤 입력을 받고, 어떤 Tool을 호출하며, 어떤 결과를 State에 추가하는지 읽어야 합니다.

4일차 연결 관점

이 관점이 있어야 4일차에서 Handoff와 Guardrail 적용 지점을 평가할 수 있습니다.

입력

Node가 받는 정보

호출

Tool 실행

결과

State에 추가

질문 유형에 따른 분기

LangGraph Routing의 핵심은 질문의 유형과 포함된 정보에 따라 다음 흐름을 다르게 선택하는 것입니다. 이런 분기 기준은 단순 편의 기능이 아니라 **Agent**의 안정성과 근거 품질을 결정하는 설계 요소입니다.

질문 유형에 따른 분기 기준

알람 코드 있음

최근 알람 이력 조회 필요 → DB/Log Tool 호출

설비 ID 있음

설비 상태 조회 가능 → DB Tool 호출

조치 절차 필요

매뉴얼 근거 필요 → search_manual Tool 호출

조회 조건 부족

설비 ID·알람 코드 없음 → 추가 질문 흐름으로 전환

 제조 현업에서는 "설비 조건을 바로 변경하라"는 위험 요청이나 조치 실행 요청은 단순 조회와 다르므로 **Guardrail** 또는 **Human Review** 단계로 보내야 합니다.

이처럼 질문 유형에 따른 분기 기준은 LangGraph Conditional Edge 설계로 연결될 수 있으며, 3일차에서는 이를 Tool 호출 조건과 State 전달 기준으로 해석합니다.

Tool Call Plan 설계

❗ 실행 후에는 agent_steps와 State를 통해 실제 계획대로 호출되었는지 확인할 수 있어야 합니다.

Tool Call Plan이란?

어떤 Tool을 어떤 순서로 호출할지 정하는 설계 메모입니다. Agent가 사용할 수 있는 Tool이 많아질수록 모든 Tool을 무조건 호출하는 방식은 적절하지 않습니다.

- 설비 상태 → 알람 이력 → 공정 상태 → 품질 지표 → 정비 이력 순차 확인
- 독립적인 조회라면 병렬 호출 전략 검토
- 기술 문서 검색은 DB 결과 확인 후 또는 초기 단계에서 함께 호출

불필요한 호출 줄이기

- 품질 영향이 질문에 없거나 line_id가 없으면 품질 지표 조회 생략
- 조회 실패 시 fallback / Saved Result Review / 사람 확인 단계 중 선택
- 위험 요청·조치 실행 요청은 Guardrail 또는 Human Review로 전달

LangGraph State와 4일차 평가 연결

State가 명확해야 Agent 간 Handoff를 평가할 수 있습니다. 4일차에는 이 정보를 기준으로 Tool 선택, 오류 처리, Guardrail 필요 지점을 평가합니다.

equipment_id / alarm_code

Tool 호출의 기준이 되는 식별자

db_results

DB/Log Tool 호출 결과를 담는 데이터 근거

rag_results

search_manual Tool이 반환한 문서 근거

agent_steps

어떤 Agent 또는 Node가 어떤 단계를 수행했는지 보여주는 실행 기록

final_summary

DB 근거와 RAG 근거를 종합한 최종 결과

State 기반 Handoff 평가

평가 질문 예시

1 CoordinatorAgent

알람 코드를 제대로 추출하지 못했는가?

2 DBInvestigationAgent

필요한 Tool을 호출하지 않았는가?

3 ManualRAGAgent

문서 검색 결과를 충분히 남겼는가?

4 IncidentSummaryAgent

근거 없는 요약을 생성했는가?

Day3 → Day4 확장

현재 Day3에서는 `agent_steps`와 결과 파일을 실행 기록으로 보고, 4일차에
서 필요하면 `mcp_call_trace.jsonl` 같은 더 정식화된 Trace 구조로 확장할
수 있습니다.



평가 가능한 실행 기록 구조를 이해하는 것이 핵심입니다.

Trace는 단순 로그가 아니다

Trace는 단순히 프로그램이 실행되었다는 로그가 아니라, Tool 호출의 증거이자 오류 분석의 출발점입니다. 제조 현장에서 Agent가 어떤 DB 근거와 어떤 문서 근거를 사용했는지 설명할 수 있어야 하므로 실행 기록은 단순 개발 로그보다 훨씬 중요한 의미를 갖습니다.

Tool 호출 증거

어떤 Tool이 어떤 입력값으로 호출되었고, 어떤 결과를 반환했는지 확인

평가·Guardrail 기준

Trace는 평가와 Guardrail의 기준이 되며, 운영·감사 관점에서도 핵심 자료

결과 전달 추적

결과가 어떤 Agent나 Summary 단계로 전달되었는지 확인 가능

Day3 Trace 해석 방법

현재 예제의 Trace 구조

현재 Day3 예제에서는 mcp_call_trace.jsonl 파일 생성을 전제로 하지 않습니다. 대신 다음 항목들을 Trace 관점으로 해석합니다.

agent_steps

db_results

rag_results

final_summary

결과 JSON/Markdown도 Trace 관점으로 해석합니다.

추적 가능성이 핵심

중요한 것은 특정 파일명이 아니라 추적 가능성입니다.

- 반복 알람 분석 Agent가 설비 상태를 조회했는가?
- 최근 알람 이력을 확인했는가?
- 기술 문서 검색 결과를 사용했는가?
- 최종 요약이 어떤 근거를 기반으로 작성되었는가?

 이 구조가 있어야 4일차에서 오류 재현, 품질 평가, Guardrail 적용 여부를 판단할 수 있습니다.

현재 예제에서 확인할 실행 기록

실행 기록을 볼 때는 결과가 생성되었는지만 확인하지 말고, 어떤 정보가 누락되었는지, 어떤 Tool 호출 결과가 최종 요약에 반영되었는지, DB 근거와 문서 근거가 구분되는지 확인해야 합니다.

1

`state['agent_steps']`

어떤 Agent가 어떤 단계를 수행했는지 보여주는 실행 순서 기록

2

`state['db_results']`

설비 상태, 알람 이력, 공정 상태, 품질 지표, 정비 이력 등 DB/Log Tool 호출 결과

3

`state['rag_results']`

search_manual Tool을 통해 검색된 기술 문서와 조치 절차 근거

4

`state['final_summary']`

DB 근거와 RAG 근거가 어떻게 종합되었는지 확인하는 기준



JSON 결과

day3_mcp_tool_agent_v2_result.json — 구조화된 실행 결과를 검토하는 데 적합



Markdown 결과

day3_multi_agent_roles_result.md — 수강생이 흐름을 읽고 설명하기에 적합

실습 예제에서 확인할 항목

Agent 실행 확인

- CoordinatorAgent가 요청 분석을 수행했는가?
- DBInvestigationAgent가 필요한 DB/Log Tool을 호출했는가?
- ManualRAGAgent가 search_manual Tool을 호출했는가?
- IncidentSummaryAgent가 결과를 종합했는가?

Tool 호출 확인

- 어떤 Tool이 호출되었는가?
- 입력값이 필요한 범위로 제한되었는가?
- 설비 ID, 알람 코드, 조회 건수, 검색어가 과도하거나 모호하지 않은가?

출력 품질 평가

- Agent가 읽기 쉬운 구조인가?
- DB 근거와 RAG 근거가 구분되어 있는가?
- 실패 또는 누락 정보가 표시되는가?

⊗ DB 조회 결과가 비어 있는데도 정상 근거처럼 요약되었거나, 기술 문서 검색 결과가 없는데 조치 절차가 확정적으로 제시되었다면 4일차 평가에서 중요한 검토 대상이 됩니다.

현업 적용에서는 같은 기준이 감사 로그, 최소 데이터 전달, Guardrail 적용 여부를 판단하는 근거가 됩니다.

Trace 관점으로 확인할 설계 질문

Tool 선택이 적절했는가?

알람 이력 확인이 필요한 상황에서 문서 검색만 수행하지 않았는지, 조치 절차가 필요한 상황에서 DB 조회만 수행하지 않았는지 확인

입력값이 과도하지 않았는가?

설비 하나에 대한 질문인데 전체 라인을 과도하게 조회하거나, 기간 제한 없이 많은 데이터 가져오는 구조라면 현업 적용 시 위험

근거 없는 요약이 없었는가?

rag_results가 비어 있는데 final_summary에서 기술 문서 근거가 있는 것처럼 설명하거나, db_results가 누락되었는데 알람 원인을 단정하는 경우

State 전달 누락이 없는가?

불필요한 Tool 호출이 없었는지, 출력 형식이 일관적인지도 확인해야 Agent가 결과를 안정적으로 해석 가능



4일차에서는 이러한 질문을 바탕으로 Routing 최적화, 오류 처리 보강, Guardrail 적용 지점을 구체적으로 검토하게 됩니다.

4일차로 연결

4일차에는 Day3에서 만든 State와 실행 기록을 기준으로 **Contract 검증**, 오류 재현, **Handoff 점검**, **Routing 최적화**, **Guardrail**과 최소 데이터 전달을 평가합니다.

1

Contract 검증

Tool 이름, 설명, 입력·출력 구조가 실제 호출 결과와 맞는지 확인

2

오류 재현

DB 조회 실패, 검색 결과 없음, 입력값 부족 같은 상황이 실행 기록에 충분히 남아 있는지 확인

3

Handoff 점검

Agent 간에 필요한 State가 누락 없이 전달되었는지 확인

4

Guardrail 평가

어떤 데이터는 최소화해야 할지, 어떤 흐름은 사람 확인으로 넘겨야 할지 판단

Day3 MCP Tool-Using Multi-Agent v2 실행 흐름

day3_mcp_tool_agent_v2.py

Day3 통합 실행 launcher로, 사용자 요청을 준비하고 Multi-Agent 실행을 호출하며, 전체 흐름이 하나의 실행 결과로 이어지도록 구성합니다.

선택 실행 명령

```
uv run python src/day3/day3_mcp_tool_agent_v2.py
```



FastMCP 기반 실행은 manufacturing_mcp_server.py가 먼저 실행되어 있어야 할 수 있으므로, 서버 준비 여부를 확인해야 합니다.

실행 흐름 단계

01

CoordinatorAgent

설비 ID와 알람 코드 추출

02

DBInvestigationAgent

DB/Log Tool 호출 → 설비 상태, 알람 이력, 공정 상태, 품질 지표, 정비 이력 확인

03

ManualRAGAgent

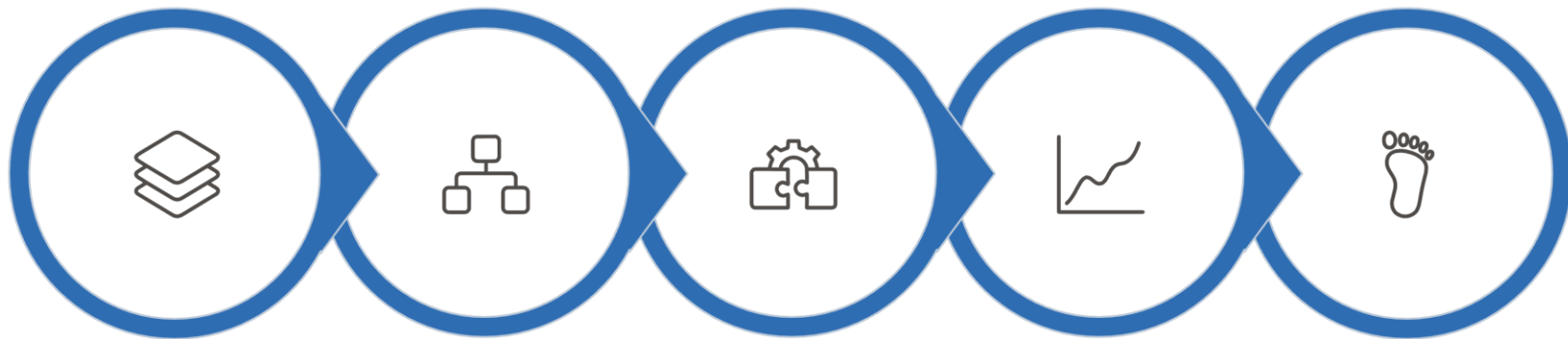
search_manual Tool 호출 → 기술 문서와 조치 절차 근거 검색

04

IncidentSummaryAgent

DB + RAG 결과 종합 → JSON/Markdown으로 저장

3일차 실습 실행 순서



Tool 분할 기준

Tool·MCP 구조

FastMCP 통합
실행

LangGraph 해
석

State·Trace 확
인

이 흐름은 단순 코드 실행이 아니라 내부 시스템 연동 구조를 Agent Tool 계층으로 이해하는 과정입니다. 실행 성공보다 구조 해석과 평가 기준 연결이 더 중요합니다.

3일차 실습 — Streamlit 실행 옵션

FastMCP 기반 실행

MCP 서버가 먼저 실행되어 있어야 합니다.

```
uv run streamlit run  
src/day3/day3_multi_agent_streamlit_app.py
```

Fallback 모드 실행

MCP 실행이 어려운 환경에서 사용합니다.

```
$env:MCP_MODE="fallback"
```

```
uv run streamlit run  
src/day3/day3_multi_agent_streamlit_app.py
```

- ❏ 통합 실행 후에는 LangGraph Routing 관점으로 흐름을 해석합니다. 어떤 조건에서 DB Tool이 호출되었는지, 어떤 조건에서 RAG Tool이 호출되었는지 확인합니다.

오늘 만든 산출물

오늘 만든 산출물은 코드 파일 자체가 아니라 **Agent** 아키텍처를 구성하는 설계 자산입니다. "파일이 생성되었는가"보다 "Tool 분할, Tool 호출 책임, State 전달, 실행 기록이 평가 가능한 형태로 남았는가"를 기준으로 확인해야 합니다.



DB/Log Tool

제조 DB와 설비 로그 기능을 Agent가 호출 가능한 형태로 감싼 구조



RAG Tool Wrapper

기술 문서 검색 기능을 Agent Tool로 표준화한 결과



FastMCP Server/Client

Tool 등록과 호출 구조를 보여주는 MCP 연결 구조



Multi-Agent 역할 매핑

어떤 Agent가 어떤 Tool을 호출하고 어떤 State를 넘기는지 정의

산출물 상세 — 설계 메모와 실행 기록

설계 메모

LangGraph Routing 설계 메모

어떤 조건에서 어떤
Tool을 호출할지 판단
하는 기준

fallback 실행 결과

MCP 실행이 어려운 상황에서도 Tool 호출 개념을 유지하는 예비 실행 구조

실행 기록 산출물

- State와 agent_steps 기반 실행 기록 — 4일차 Handoff와 Guardrail 평가 입력 자료
- day3_mcp_tool_agent_v2_result.json — 구조화된 실행 결과 검토용
- day3_multi_agent_roles_result.md — 흐름을 읽고 설명하기 위한 산출물

✓ 이 산출물들은 단순 실행 결과가 아니라 4일차 평가와 5일차 적용 전략의 기반입니다.

3일차 마무리 메시지

3일차의 핵심은 DB 연결이 아니라 Tool 설계입니다. MCP는 내부 시스템을 Agent가 안전하게 호출하게 하는 표준 계층입니다.

MCP

Tool의 규격과 연결 방식 정리 — 이름, 설명, 입력·출력 구조, 오류 처리 기준

LangGraph

MCP Tool을 어떤 조건과 순서로 호출할지 설계하는 흐름 제어 구조

Multi-Agent

Tool 호출 책임과 State 전달을 명확히 하기 위한 구조

Day4 · Day5 연결 전략

4일차 평가 항목

- Tool 선택이 적절했는가?
- DB 근거와 RAG 근거가 구분되었는가?
- 오류와 누락 정보가 남았는가?
- Guardrail이 필요한 지점은 어디인가?

4일차에는 오늘 남긴 State와 실행 기록을 기준으로 품질과 안정성을 평가합니다.

5일차 확장 전략

5일차에는 이 구조를 통합 Agent와 사내 PoC 전환 전략으로 확장합니다.



☑ 따라서 오늘의 산출물은 단순 실습 결과가 아니라 4일차 평가와 5일차 적용 전략의 기반입니다.